1. Individual Progress Update

1.1 Team Member 1: Younghak Yoo

Scheduled Items

[Weeks 1 - 5]: Facilitated project ideation and brainstorming sessions; established the global design system, core UI/UX wireframes, and initial layout architectures for the image-based search dashboard.

[Weeks 6 - 9] Engineered the initial frontend prototypes for the Search engine pipeline; finalized comprehensive feature specifications for the Gallery and Journal modules; designed domain-specific data transfer objects and migrated components to a modular directory layout.

[Week 10] Led the agile scope realignment and resource reallocation following team restructuring; conducted extensive UI refactoring to smoothly deprecate the Travelize feature without causing systemic regressions.

[Weeks 11 - 13] Managed the frontend migration from a single-tier look-up scheme to a high-precision parallel multi-tier pipeline; optimized asynchronous view re-rendering states; designed and implemented the unified Journal generation canvas utilizing combined CLIP and OpenAI endpoints.

[Weeks 14 - 15] Implemented interactive micro-calibration tools (Inline text edits and Map-pin adjustments); completed the Profile and Settings synchronization views; coordinated end-to-end cross-verification test suites and finalized the production production release.

Completed Items

1) Core Architecture: 19-Stage Location Search Pipeline

- **Phase 1: Input & Preprocessing (Stages 1–4)**
- **Image Validation & Loading UI**: Validates image file size and structure to block corrupted files, while triggering a smooth flight-animation screen to handle processing wait times.
- **EXIF Check & Dual-Stream Forking**: If EXIF GPS data is found, the pipeline instantly completes (short-circuit). If data is missing, the image splits into two tracks: a raw stream for web/AI detection and an enhanced copy with corrected brightness, blur, and angles.
- **Phase 2: Parallel Analysis & Data Cleansing (Stages 5–8)**
- **Multi-API Signal Harvesting**: Runs Google Vision OCR, Logo, Web Detection, and Landmark APIs at the same time to gather all possible visual clues.
- **Smart Data Filtering**: Instantly drops useless text fragments, broken URLs, and random string noise to keep only clean location data.
- **Phase 3: Spatial Consolidation & Scoring (Stages 9–12)**

- **Location Grouping**: Combines loose clues using Google Place IDs, a 1km coordinate radius, or matching names. Automatically absorbs larger regional zones into specific landmark points.
- **Weighted Scoring & Consensus Bonus**: Calculates scores using custom source weights (GPS: 1.0, Landmark/AI: 0.75, Web/Logo: 0.70, OCR: 0.50). Gives a bonus score when completely different APIs agree on the exact same place.
- **Verdict Classification**: Organizes final results into four clear confidence classes: *Confident*, *Likely*, *Suggestions*, or *Failed*.

- **Phase 4: User Context & AI Refinement (Stages 13–16)**

- **Dynamic Hint Scaling**: Boosts scores for candidates matching user-input hints (Country x1.3, City x1.2) and penalizes conflicting ones (Country x0.4, City x0.5).
- **Smart Cost-Saving Fallback**: Instantly locks the result and skips expensive AI calls if an exact web match is found. If still uncertain, it invokes a Vision LLM (GPT) to add missing clues and runs the scoring cycle one final time.

- **Phase 5: Session Cooperation & Manual Adjustments (Stages 17–19)**

- **Multi-Photo Cluster Sync**: For bulk uploads, calculates a central travel zone using high-confidence photos (score $\ge$ 0.5) to fix and fill in missing location data on surrounding photos.
- **Interactive UX Calibration**: Gives users direct control to inline-edit business names or manually adjust location markers on a live map tool (*Pick on Map*).

2) Journal Generation & Visual Analytics (100% Completed)

- **Multi-Modal Tagging**: Uses the CLIP API to automatically analyze and extract context tags (subjects, moods, and activities) from saved travel photos.
- **Generative Journal Canvas**: Combines resolved location metadata and CLIP tags within OpenAI to automatically write natural, editable travel log drafts.
- **Visual Dashboard Analytics**: Aggregates tag history to generate interactive data charts, helping users visualize their overall travel patterns, themes, and habits.

3) Gallery & Testing (100% Completed)

- **Relational Database Sync**: Maps all verified search data, coordinates, and manual user overrides into PostgreSQL (`uploaded_images` and `searches` tables), displaying them on a performant, lazy-loaded grid view.
- **End-to-End QA Testing**: Follows the official Bug Tracking Guide to run cross-verification tests on features built across the team. Catches and logs tough edge cases—such as scrubbed metadata or blank images—on GitHub Issues with accurate severity tags until they are 100% resolved.

Partially Completed Items

- **None**

1.2 Team Member 2: Jaemin Jeon

Scheduled Items (Based on Project Timeline)

[Weeks 1 - 5] Conducted project ideation and brainstorming sessions; developed user profile wireframes, ancillary feature flowcharts, and the global application layout layout.

[Weeks 6 - 9] Defined core technical specifications for the real-time communication modules; engineered backend API routers and frontend view wrappers for the Profile and Settings screens.

[Week 10] Contributed to strategic resource reallocation and codebase refactoring during team reorganization, ensuring smooth component detachment after the deprecation of the Travelize module.

[Weeks 11 - 13] Reconfigured the chat service into an optimized, context-mapped communication hub; designed WebSocket persistent connection states paired with stateless REST fallback endpoints.

[Weeks 14 - 15] Developed the administrative oversight interface with secure authorization filters; executed comprehensive cross-functional repository merging, end-to-end integration tests, and production validation.

Completed Items (100% Finished)

1) User Profile and Settings (100% Completed)

- **Profile Orchestration & State Sync**: Built a modular Profile management screen, enabling users to dynamically fetch, modify, and persist personal identity records. Integrated secure RESTful handlers to instantly synchronize state updates with the underlying PostgreSQL account tables.
- **Preference Controls & Context Providers**: Developed an elegant configuration view allowing on-the-fly mutations of display preferences, interface themes (Dark/Light mode context states), and data privacy constraints, providing a complete account management ecosystem.

2) Centralized Admin Control Panel & Access Control (100% Completed)

- **Admin Dashboard**: Created a secure Admin Panel where authorized managers can easily check user registrations, reported content, and system data summaries.
- **User Access Separation**: Added strong route guards and token validation on both frontend and backend to keep admin features completely separate from normal traveler accounts, preventing unauthorized access.

3) Tag-Based Real-Time Chat Infrastructure (100% Completed)

- **Travel Lounges**: Changed the general chat into 13 specific travel lounges (such as *Food & Cafe*, *Historical & Heritage*, *Urban & Street*, *Beach & Coast*, and *Sunset & Sunrise*). This connects with the photo search feature by automatically guiding users to chat rooms that match their photo's AI tags.
- **Real-Time Chat & Message History**: Built high-performance, real-time messaging using WebSockets. Also implemented a reliable backup system using standard REST APIs to load past messages from the database whenever a user enters a room or refreshes the page.

4) Code Integration & Documentation(100% Completed)

- **Code Merging & Integration**: Worked with the team leader to fix merge conflicts, data alignment, and CORS issues. Successfully combined the frontend and backend into a clean, container-ready main branch.
- **Project Documentation**: Wrote and updated core project guides, including the README setup instructions, API routing sheets, deployment checklists, testing strategies, and the Bug Tracking Guide.

Partially Completed Items

- **None**

Group Progress Update

1. Overall Project Progress

Feature	Assigned To	Status
Project setup and infrastructure	Both	✅ Complete
Authentication (register / login / OTP)	Younghak	✅ Complete
Image upload and EXIF extraction	Younghak	✅ Complete
Location Search pipeline (multi-tier)	Younghak	✅ Complete
Gallery system	Younghak	✅ Complete
Journal generation and analytics	Younghak	✅ Complete
Profile and Settings	Jaemin	✅ Complete

Tag-based Live Chat (13 lounges)	Jaemin	✅ Complete
Admin Control Panel	Jaemin	✅ Complete
Codebase integration and CORS	Jaemin	✅ Complete
Deployment (Railway + Vercel)	Both	✅ Complete
Documentation and README	Jaemin	✅ Complete

Overall, our team is on schedule. All core use cases defined in the original Software Design Document have been implemented and integrated into the main branch. The project is fully deployed and accessible via a live link. Despite a mid-semester team restructuring that required significant replanning, every feature within the revised scope has been delivered by the beta release milestone.

2. Adjustments from the Original Plan

1. Removal of the Travelize feature

The most significant change from the original plan was the removal of the Travelize itinerary generation feature, which resulted from a team member leaving the project mid-semester. At the time, Travelize was one of the primary planned features. Rather than attempting to maintain an understaffed scope, the team made the decision to redirect all resources toward delivering the remaining features at a higher level of quality. This decision was discussed with the instructor and reflected in the revised Software Design Document.

2. Search pipeline architecture change

The original Search design used a sequential single-path approach — each API would be called one at a time, and if a result was found, the pipeline would stop. While simple, this approach had a critical weakness: a single wrong early result would produce an incorrect final answer with no way to verify it.

After the scope change, we redesigned the pipeline using a signal fusion architecture. Instead of stopping at the first result, all available APIs run in parallel and each result is assigned a

confidence-weighted score. Candidates are ranked by how much independent evidence supports them. The pipeline only moves to a more expensive API layer (such as GPT Vision) if the current evidence is insufficient to reach a Confident verdict. This approach produces more reliable results, avoids unnecessary API costs, and gives users a ranked candidate list instead of a single unverified answer.

3. Live Chat redesigned into tag-based travel lounges

The original chat design was a general-purpose message board where users could talk to each other without a clear purpose. After the scope change, we reconsidered what would actually be useful to travelers using this service.

The redesigned system replaced the general chat with 13 permanent tag-based travel lounges, including channels such as Food & Cafe, Historical & Heritage, Urban & Street, Beach & Coast, and Sunset & Sunrise. Each lounge maps directly to the tags that the Search pipeline extracts from uploaded photos. When a user completes a search, the system automatically highlights the lounges that match their photo's tags, so users can immediately connect with other travelers who share the same interests.

This change gave the chat feature a clear purpose: it is no longer just a place to talk, but a direct extension of the search experience that connects people through shared travel contexts.

4. Core user loop

These three changes together shaped what we consider the central achievement of this project: a complete and coherent core user loop.

Upload photo
↓
Location candidate extraction (Search)
↓
Save results and manage gallery
↓
Generate and edit travel journal
↓
View travel analytics and get AI destination recommendations
↓
Join a relevant traveler lounge (Live Chat)

Every feature in the current scope serves a specific role in this loop. No feature exists in isolation. Our team focused all efforts on ensuring this loop works end-to-end without gaps, and

we believe that cohesion is what distinguishes this project from a collection of unconnected features.

3. Issues and Difficulties

1. Team restructuring mid-semester

The most disruptive event was losing a team member partway through development. This required us to immediately re-evaluate which features were feasible with two people, redistribute responsibilities, and rewrite portions of the design document. The period immediately after the restructuring was the most difficult, as we had to make scope decisions quickly while continuing active development.

Resolution: We prioritized completing the core user loop over recovering the removed features. Once the revised scope was fixed, development proceeded without further major disruptions.

2. Search pipeline complexity

The multi-tier signal fusion pipeline turned out to be significantly more complex to implement than initially estimated. Normalizing outputs from five different Google Vision detectors into a single scoring system required extensive testing, especially for edge cases such as images with no detectable landmarks, images where OCR and Landmark results pointed to different locations, and images where all detectors returned empty results.

Resolution: We built a dedicated signal normalization layer and tested each edge case individually using a set of manually curated test images. All critical edge cases are now handled with defined fallback behavior.

3. WebSocket and REST fallback coordination

Implementing reliable real-time messaging required careful coordination between the WebSocket connection manager and the HTTP fallback polling system. Early in development, there were cases where messages sent through the fallback path were duplicated when the WebSocket reconnected.

Resolution: Message deduplication was handled by using the database record ID as a unique identifier on the frontend, preventing duplicate message cards from rendering even when the same message arrived through both channels.

4. Codebase integration and CORS

Merging the Search and Chat feature branches revealed several conflicts in the Alembic migration chain and mismatched API response schemas between frontend and backend. CORS configuration also required adjustment when the deployed Vercel domain differed from the local development URL.

Resolution: All migration files were reordered into a clean sequential chain, response schemas were aligned using the agreed API reference document, and CORS was configured to accept only the production Vercel domain in the deployed environment.

4. Progress Grade

Our team self-assesses our overall progress as: A−

We arrived at this grade for the following reasons.

Reasons for the high mark:

When this project was first proposed, the goal was straightforward: build a service that helps users find the location of a travel photo. What we ended up building goes considerably further than that. The service now covers the full lifecycle of a travel memory — from finding where a photo was taken, to organizing it in a personal gallery, to generating a written journal entry, to visualizing travel patterns and receiving AI-based destination recommendations, to connecting with other travelers in topic-specific lounges. Every step in that flow is implemented, connected, and working in the deployed application.

The Search pipeline in particular evolved well beyond the original plan. The multi-tier signal fusion architecture with confidence-weighted scoring, user hint integration, multi-photo session clustering, and manual calibration tools represents a meaningfully more reliable system than what was initially designed. This was not a simple feature to build, and the fact that it works across a wide range of edge cases reflects a significant amount of careful engineering work.

The core user loop is coherent. Each feature feeds the next, and the overall experience has a clear purpose for the target user: a traveler who wants to do more with their photos than just store them.

Reason for the deduction:

The team restructuring mid-semester caused a period of uncertainty that affected our development rhythm. The compressed timeline that followed meant some features did not receive the level of polish we originally intended. There are UI details and edge case refinements that we would have addressed with more time. This is the honest reason we do not assign a full A.

5. Work Strategy Going Forward

Remaining work before the final release:

- Address all open bugs logged on GitHub Issues from the beta review period
- UI polish pass: consistent spacing, loading states, and error messages across all pages
- Seed the deployed database with demonstration data (3 test accounts, pre-analyzed images, sample journals and chat messages) for the final presentation
- Prepare the final demo script covering the complete core user loop end-to-end
- Update the API documentation to reflect any endpoint changes made since the last revision
- Confirm the deployed application (Railway + Vercel) remains stable and accessible throughout the final presentation period

Testing status:

All core use cases have been manually tested end-to-end. Edge cases identified during QA are logged on GitHub Issues with severity labels. All critical and major bugs have been resolved. Minor cosmetic issues and non-blocking edge cases are scheduled for resolution before the final release date.